Software Design Description 120

CAM to DevKit SPI Interface

Overview

This document describes the SPI DMA link between the ESP32-CAM (master) and the ESP32-DevKitV1 (slave). This is the primary data path for raw grayscale frames flowing from camera to main controller, and the command/timestamp path flowing from main controller back to camera.

The link is bidirectional: MOSI carries frame data (CAM to DevKit), MISO carries commands and timestamps (DevKit to CAM). Both directions share a single SPI bus – full duplex is available but not required since command traffic is low-bandwidth and bursty.

> **CAM side:** `DMA_SPI_master.c` (SPI master) | **DevKit side:** `cam_spi.c` (SPI slave)

Physical Layer

Parameter	Value
SPI Host	HSPI_HOST
Clock	18 MHz
Mode	0 (CPOL=0, CPHA=0)
MOSI	GPIO 13 (CAM TX, DevKit RX)
MISO	GPIO 12 (DevKit TX, CAM RX)
SCLK	GPIO 14
CS	GPIO 15
Max DMA transfer	4092 bytes
DMA channel	Auto

Pin conflict: GPIO 13 is shared with the SD card (SDMMC DATA0). SPI and SD cannot operate simultaneously. This is resolved by design: SD is only accessed during the post-capture phase when SPI is idle. No runtime pin swapping occurs during capture.

MOSI: Frame Data (CAM to DevKit)

Frame Protocol

Each frame transmission consists of a 12-byte header followed by raw pixel data in 4092-byte DMA chunks.

Header (12 bytes):

Offset	Size	Field	Value
0	4	Magic	`0xCAFEBEEF`
4	4	Size	Frame size in bytes (76800 for QVGA)
8	4	Checksum	XOR of all image bytes

Payload: Raw grayscale pixels, row-major, 320x240 = 76,800 bytes. Transmitted in ceil(76800/4092) = 19 DMA chunks.

Transfer Timing

Metric	Value
Frame size	76,800 bytes + 12 byte header
SPI clock	18 MHz
Transfer time	~34 ms per frame
Target rate	30 fps (33 ms period)
Utilization	~100% — SPI is the bottleneck at 30fps

At 18 MHz, the SPI bus is nearly saturated at 30 fps. Actual capture rate may settle at 25-28 fps depending on DMA overhead and task scheduling.

Bus Mutex

`spi_dma_transmit()` holds a FreeRTOS mutex for the full duration of each transfer. `spi_dma_deinit()` blocks on this mutex, ensuring no in-flight transactions when the bus is torn down for post-capture SD access.

MISO: Commands (DevKit to CAM)

Wake Handshake

On PIR trigger, the DevKitV1 wakes the CAM and sends a handshake packet:

Offset	Size	Field	Description
0	4	Magic	`0xDEADBEEF`
4	4	Command	`0x01` = START_CAPTURE
8	4	Epoch	UTC timestamp (seconds since Unix epoch) from RTC

The CAM stores the epoch and starts `esp_timer` as a local time baseline. All subsequent timestamps are derived as `epoch + esp_timer_get_time() / 1e6`.

Stop Command

When the DevKitV1 determines the motion event is over:

Offset	Size	Field	Description
0	4	Magic	`0xDEADBEEF`
4	4	Command	`0x02` = STOP_CAPTURE

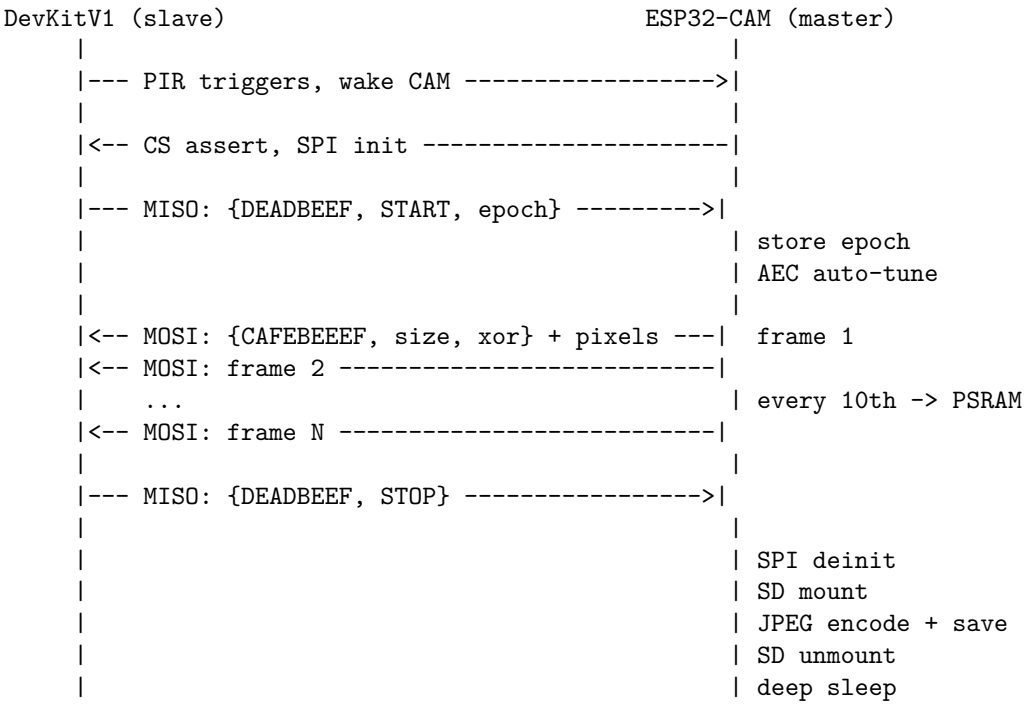
On receiving STOP, the CAM: 1. Drains the SPI transmit queue 2. Deinits SPI DMA 3. Enters post-capture phase (JPEG encode + SD save) 4. Returns to deep sleep

Implementation

MISO commands are received during the full-duplex SPI transaction. The CAM checks the rx_buffer after each `spi_device_transmit()` for a valid magic word. When no command is pending, the DevKit slave holds MISO at 0x00 (no-op).

This avoids dedicated command transactions — commands piggyback on frame data transfers with zero overhead.

Sequence Diagram



DevKit Slave Side

The DevKitV1 receives frames via `cam_spi.c` (SPI slave):

- Validates magic word `0xCAFEBEEF`
- Checks XOR checksum
- Enqueues frame to `img_proc` task via FreeRTOS queue
- Drops frames on checksum mismatch (logged, not retransmitted)

MISO command injection: the slave pre-loads the TX buffer with the next command (or 0x00 no-op) before each transaction. The ESP-IDF SPI slave driver handles simultaneous TX/RX.

Error Handling

Condition	Behavior
Checksum mismatch	Frame dropped, logged on DevKit
SPI timeout	CAM logs warning, retries next frame
Bus mutex timeout	Frame dropped (SPI busy with previous transfer)
No wake command	CAM remains in deep sleep
No stop command	CAM captures until PSRAM save buffer is full, then auto-stops

---

Source Files

File	Side	Purpose	Status
`esp32cam/src/DMA_SPI_master.c`	CAM	SPI DMA master, frame TX, command RX	Done (needs MISO rx + 18MHz)
`esp32cam/include/DMA_SPI_master.h`	CAM	Pin defines, clock config	Done (needs clock bump)
`esp32_devkitv1/main/spi_slave_interprocessor/cam_spi.c`	DevKit	SPI slave, frame RX, command TX	Done (needs MISO command inject)

References

- SDD100 — ESP32-CAM Overview
- SDD200 — ESP32-DevKitV1
- ESP-IDF SPI Master Driver
- ESP-IDF SPI Slave Driver